# Séance 1

## Présentation du langage + atelier d'installation

### Objectifs de la séance

- Situer Python : histoire, versions, usages
- Utiliser l'interpréteur interactif (REPL) et la documentation officielle
- Installer un environnement de travail complet : Python, IDE, venv, JupyterLab
- Écrire et exécuter un premier script et un premier notebook

# Plan de la séance

| Durée | Bloc |
|---|---|
| 10 min | Accueil et tour de table |
| 20 min | Théorie 1 : histoire, Python 2 vs 3, REPL, documentation, IDE |
| 30 min | 🧪 Atelier : installation + venv + premier script + premier notebook |
| 5 min | Pause |
| 20 min | Théorie 2 : instructions, commentaires, premières variables |
| 30 min | 🧪 Lab partie 2 + étape 0 du projet GestParc |
| 5 min | Récapitulatif |

# Python en quelques dates

| Année | Événement |
|---|---|
| 1989 | Guido van Rossum commence Python (nom tiré des *Monty Python*) |
| 1991 | Première version publique (0.9.0) |
| 2000 | Python 2.0 |
| 2008 | Python 3.0 : rupture volontaire de compatibilité |
| 2018 | Guido quitte le rôle de « BDFL », gouvernance par un *Steering Council* |
| 2020 | Fin de vie de Python 2 (dernière version : 2.7.18) |
| Depuis 2019 | Une nouvelle version 3.x chaque année en octobre (PEP 602) |

Nous travaillons en Python 3.12 ou plus récent.

# Pourquoi Python ?

- **Lisible** : la syntaxe ressemble à du pseudo-code, l'indentation fait partie du langage
- **Polyvalent** : un même langage pour des besoins très différents
- **Batteries included** : une bibliothèque standard très riche
- **Écosystème** : plus de 500 000 paquets sur PyPI

| Profil | Usage typique |
|---|---|
| Admin sys / réseau | Automatisation, Ansible, analyse de logs |
| Data | pandas, NumPy, Jupyter, machine learning |
| Développeur | Web (Django, FastAPI), outils, tests |
| Chef de projet | Scripts de reporting, extraction de données |

# Python 2 vs Python 3 : les différences visibles

| | Python 2 | Python 3 |
|---|---|---|
| Affichage | `print "Bonjour"` (instruction) | `print("Bonjour")` (fonction) |
| Division | `7 / 2` → `3` | `7 / 2` → `3.5` |
| Texte | ASCII par défaut | Unicode par défaut |
| Saisie | `raw_input()` | `input()` |
| Statut | **Fin de vie depuis 2020** | Version actuelle |

⚠️ Un tutoriel ou une réponse en ligne qui utilise `print "..."` sans parenthèses est **obsolète**.

# L'interpréteur interactif (REPL)

**R**ead – **E**val – **P**rint – **L**oop : on tape, Python évalue et affiche.

```
$ python
Python 3.12.7 (main, ...) on linux
>>> 2 + 3
5
>>> 7 / 2
3.5
>>> "Gest" + "Parc"
'GestParc'
>>> exit()
```

- Windows : `py` ou `python` · Linux / macOS : `python3`
- Quitter : `exit()` ou Ctrl+D (Linux/macOS), Ctrl+Z puis Entrée (Windows)

# La documentation officielle

- **python.org** : téléchargements, actualités
- **docs.python.org/fr/3/** : documentation en français
  - *Tutoriel* : à lire en complément de la formation
  - *Référence de la bibliothèque standard* : à garder sous la main
- Dans le REPL :

```
>>> help(print)      # aide sur une fonction
>>> import this      # le Zen of Python (on y reviendra en S14)
```

💡 Réflexe à prendre : **la documentation officielle avant le forum.**

# Bien choisir son environnement de développement

| Outil | Points forts | Pour qui ? |
|---|---|---|
| **VS Code** + extensions Python et Jupyter | Léger, gratuit, multi-langages, notebooks intégrés | Tous (outil de référence de la formation) |
| **PyCharm** | Très complet, refactoring, debugger avancé | Développeurs |
| **JupyterLab** | Code + texte + graphiques, exécution par cellules | Data, exploration, labs |
| IDLE | Fourni avec Python | Dépannage seulement |

Nos labs : **notebooks** `.ipynb` · nos programmes : **scripts** `.py`

# L'environnement virtuel (venv)

Un **venv** = une copie isolée de Python, propre à un projet.

```
gestparc/
├── .venv/          ← Python + paquets du projet (ne pas versionner)
├── main.py
└── README.md
```

- Évite les conflits de versions entre projets
- Ne touche pas au Python du système (crucial sur Linux)
- Se crée en une commande : `python -m venv .venv`

# 🧪 À vous : atelier d'installation (30 min)

Suivez le notebook **`S01_lab_enonce.ipynb`**, partie 1, ou la fiche ci-dessous :

```
# 1. Vérifier Python (3.12 ou plus)
python --version              # Linux/macOS : python3 --version
# 2. Créer le projet et son venv
mkdir gestparc && cd gestparc
python -m venv .venv
# 3. Activer le venv
source .venv/bin/activate     # Windows : .venv\Scripts\activate
# 4. Installer JupyterLab
python -m pip install jupyterlab
# 5. Lancer le contrôle d'installation
python check_install.py
```

# Instructions et exécution d'un script

Un script s'exécute **de haut en bas, une instruction par ligne**.

```
# premier_script.py
print("Inventaire GestParc")
print("Nombre de postes :", 42)
print(10 + 32)
```

```
$ python premier_script.py
Inventaire GestParc
Nombre de postes : 42
42
```

`print()` accepte plusieurs valeurs séparées par des virgules : elles sont affichées séparées par un espace.

# Les commentaires

```
# Un commentaire commence par # et va jusqu'à la fin de la ligne
print("Démarrage")   # commentaire en fin de ligne

# Bon commentaire : il explique POURQUOI
# On garde 10 % de marge pour les mises à jour système.
reserved_gb = 50

# Mauvais commentaire : il répète le code
x = 50  # x vaut 50
```

Règle de la formation : **code et noms en anglais, commentaires en français**.

# Premières variables

Une variable est **un nom qui désigne une valeur**. On la crée par affectation avec `=`.

```
hostname = "srv-web-01"
cpu_count = 8
ram_gb = 31.4
is_online = True

print(hostname, "a", cpu_count, "cœurs")
```

```
srv-web-01 a 8 cœurs
```

Pas de déclaration de type : Python le déduit de la valeur (détails en S2).

# Python comme calculatrice

| Opérateur | Sens | Exemple | Résultat |
|---|---|---|---|
| `+` `-` `*` | addition, soustraction, multiplication | `8 * 4` | `32` |
| `/` | division (toujours décimale) | `7 / 2` | `3.5` |
| `//` | division entière | `7 // 2` | `3` |
| `%` | reste (modulo) | `7 % 2` | `1` |
| `**` | puissance | `2 ** 10` | `1024` |

```
total_gb = 512
used_gb = 389
free_gb = total_gb - used_gb      # 123
```

# Nommer ses variables

| ✅ Correct | ❌ Incorrect | Pourquoi |
|---|---|---|
| `server_count` | `2nd_server` | Ne commence pas par un chiffre |
| `disk_usage` | `disk-usage` | `-` est l'opérateur de soustraction |
| `vat_rate` | `class` | Mot réservé du langage |
| `ticket_id` | `t` | Nom non explicite |

- Convention **snake_case** : minuscules et `_`
- Python **distingue les majuscules** : `Total` et `total` sont deux variables
- Constantes en MAJUSCULES par convention : `APP_NAME = "GestParc"`

# Pièges fréquents de la première soirée

```
>>> print "Bonjour"
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?

>>> print(nb_servers)
NameError: name 'nb_servers' is not defined

>>> Hostname = "srv-01"
>>> print(hostname)
NameError: name 'hostname' is not defined. Did you mean: 'Hostname'?
```

💡 **Lire le message d'erreur en partant de la dernière ligne** : type de l'erreur + explication.

## 🧪 À vous : Lab partie 2 + projet fil rouge (30 min)

Notebook `S01_lab_enonce.ipynb`, parties 2 et 3

- Exercices ⭐ à ⭐⭐⭐ : variables, calculs, nommage, lecture d'erreurs
- Exercice de synthèse : estimer la durée d'analyse d'un fichier de logs
- **GestParc – étape 0** : dossier projet, venv, `README.md`, `main.py` qui affiche la bannière de l'application

Les cellules `assert` vérifient vos réponses : **aucun message = c'est juste**.

# Récapitulatif

- Python 3 uniquement : Python 2 est en fin de vie depuis 2020
- REPL pour tester, **script** `.py` pour les programmes, **notebook** pour explorer
- Un **venv par projet** : `python -m venv .venv` puis activation
- `print()` affiche, `#` commente, `=` affecte une valeur à un nom
- Noms en snake_case, explicites, en anglais

**Pour la prochaine fois** (optionnel) : lire les sections 1 à 3 du tutoriel officiel sur docs.python.org/fr/3/

**Séance 2** : types, séquences, dictionnaires, références et mutabilité